

ITI123 Final Project Report

AI-Based Badminton Shot Classification: From Pose-Only Failure to Video-Based Success

Paul George Karippaparambil

Student ID: 8031408E

Email: paulgeorge@nus.edu.sg

ITI123 Generative AI & Deep Learning Project

February 06, 2026

Abstract

This project explores automated badminton shot classification using deep learning, addressing the challenge of providing accessible technique feedback for recreational players. We implemented and rigorously evaluated three progressive approaches: (1) **pose-only skeleton-based models** achieving 50% accuracy on Clear vs Smash binary classification, demonstrating fundamental limitations of body pose features alone; (2) **video-based ResNet18+BiLSTM architecture** achieving 74.6% accuracy on 5-class shot type classification, representing a 49% relative improvement; and (3) **Shot Coach web application** providing real-time analysis with intelligent video metadata tracking and user guidance.

Key contributions include: (a) empirical demonstration that skeletal pose data is insufficient for overhead shot discrimination due to identical body positions at contact; (b) successful transfer learning from ImageNet to badminton domain using ResNet18 backbone; (c) effective handling of severe class imbalance (3.5% to 31.8%) using Focal Loss; (d) deployment of production-ready application with transparency features addressing AI governance principles. The system processes 2-3 second video clips in 5-10 seconds and provides actionable feedback through an intuitive Streamlit interface. This work demonstrates the complete machine learning lifecycle from problem formulation through failed experiments to successful deployment, emphasizing the importance of iterative development and user-centered design.

Contents

1	Introduction	3
1.1	Problem Statement and Motivation	3
1.2	Project Objectives	3
1.3	Shot Type Taxonomy	3
1.4	Dataset: ShuttleSet	4
1.5	Success Criteria and Evaluation Metrics	4
2	Phase 1: Pose-Only Classification (Failed Approach)	5
2.1	Motivation and Hypothesis	5
2.2	Methodology	5
2.2.1	Dataset Preparation	5
2.2.2	Pose Extraction Pipeline	6
2.2.3	Feature Engineering	6
2.2.4	Data Splitting Strategy	7
2.2.5	Sequence Padding and Normalization	8
2.2.6	Model Architectures	8
2.3	Results: Complete Failure	9
2.3.1	Baseline Model Performance	9
2.3.2	Enhanced Architecture Attempts	10
2.4	Root Cause Analysis	11
2.4.1	Hypothesis Testing: Wrist Angle Analysis	11
2.4.2	Complete Body Pose Comparison at Contact	11
2.4.3	Missing Critical Information	12
2.5	Lessons Learned from Phase 1 Failure	12
3	Phase 2: Video-Based Classification (Successful Approach)	13
3.1	Motivation for Architectural Pivot	13
3.2	Architecture Design: ResNet18 + BiLSTM	14
3.2.1	Overall Pipeline	14
3.2.2	Spatial Feature Extraction: ResNet18	14
3.2.3	Temporal Modeling: Bidirectional LSTM	15
3.2.4	Complete Model Architecture	16
3.3	Dataset Expansion to 5-Class Problem	17
3.3.1	Data Statistics	17
3.3.2	Class Imbalance Mitigation: Focal Loss	18
3.4	Training Strategy and Configuration	18
3.4.1	Two-Stage Training	18
3.4.2	Complete Training Configuration	20
3.5	Results and Comprehensive Evaluation	21
3.5.1	Overall Performance Metrics	21
3.5.2	Per-Class Performance Analysis	22
3.5.3	Confusion Matrix Analysis	23
3.5.4	Training Dynamics Over Time	24

3.6	Ablation Studies	25
4	Phase 3: Shot Coach Application Deployment	26
4.1	Motivation: From Research to Practice	26
4.2	System Architecture	27
4.3	Core Components	27
4.3.1	Shot Classifier Module	27
4.3.2	Video Metadata Tracking (Key Innovation)	28
4.4	Smart Warning System	29
4.5	Camera Angle Robustness Analysis	30
4.5.1	Training Data Camera Characteristics	30
4.5.2	Expected Performance by Camera Angle	30
4.5.3	User Guidance in Application	31
4.6	Application Features Summary	32
4.7	Limitations and Considerations	32
5	Comparative Analysis: Pose-Only vs Video-Based	34
5.1	Quantitative Comparison	34
5.2	Qualitative Insights	34
5.3	Key Lessons Learned	35
6	AI Governance and Ethical Considerations	36
6.1	Transparency and Explainability	36
6.2	Privacy and Data Protection	37
6.3	Safety and Robustness	38
6.4	Fairness and Inclusivity	39
6.5	Accountability and Governance	40
6.6	AI Governance Checklist	40
7	Conclusion	41
7.1	Project Achievements	41
7.2	Lessons Learned	42
7.3	Project Impact	43
7.4	Future Research Directions	43
7.5	Final Remarks	45
A	Technical Specifications	47
A.1	Hardware and Software Environment	47
A.2	Code Repository Structure	47
A.3	Deployment Commands	48
A.3.1	Phase 1: Pose-Only Training	48
A.3.2	Phase 2: Video-Based Training	49
A.3.3	Phase 3: Shot Coach Deployment	49
A.4	Sample Application Output	50
A.4.1	Shot Coach Web Interface Output	50

1 Introduction

1.1 Problem Statement and Motivation

Badminton technique development traditionally requires regular access to professional coaching, which presents barriers including scheduling constraints, geographic limitations, and financial costs. As a recreational badminton player, I identified this accessibility gap as an opportunity for AI-powered automation. This project investigates whether deep learning can provide reliable shot classification and technique analysis, democratizing access to coaching feedback.

Core Challenge: Badminton shots exhibit subtle differences in execution despite similar body movements. For example, Clear and Smash shots—both overhead strokes—differ primarily in racket angle and follow-through rather than body pose. This presents a non-trivial classification problem requiring careful feature selection and model design.

1.2 Project Objectives

This project pursues three progressive objectives aligned with the machine learning development lifecycle:

Objective 1: Investigate pose-based classification feasibility (Phase 1)

- Research Question: Can skeletal keypoint sequences distinguish Clear vs Smash?
- Hypothesis: Wrist angle and arm extension capture shot-specific patterns
- Expected Outcome: Binary classifier with >80% accuracy

Objective 2: Develop video-based multi-class classifier (Phase 2)

- Research Question: Can visual appearance features improve classification across all 5 shot types?
- Approach: Transfer learning with ResNet18+BiLSTM architecture
- Target: >70% accuracy on 5-class problem

Objective 3: Deploy user-facing application (Phase 3)

- Research Question: How to make trained model accessible to non-technical users?
- Requirements: Web interface, real-time inference, explainable predictions
- Success Criterion: Functional tool providing actionable feedback

1.3 Shot Type Taxonomy

Classification targets five fundamental badminton strokes based on international coaching standards:

Table 1: Badminton Shot Type Characteristics

Shot Type	Technical Characteristics	Tactical Purpose
Clear	High trajectory (30–45° upward), contact overhead, defensive arm extension	Push opponent to back court, buy recovery time
Drive	Flat trajectory (0–10°), fast speed (>250 km/h), parallel to ground	Quick attack, maintain pressure
Drop	Soft touch, minimal trajectory height, just clears net	Force opponent forward, create court space
Lift	High defensive lob (40–60°), from low court position	Defensive recovery, regain court position
Smash	Steep downward angle (-20–0°), maximum power (>350 km/h)	Primary attacking shot, score points

Classification Difficulty: Drive vs Lift and Clear vs Smash pairs are particularly challenging due to similar execution patterns but different tactical intentions.

1.4 Dataset: ShuttleSet

All experiments utilize the ShuttleSet dataset [1], a gold-standard resource for badminton video analysis:

Table 2: ShuttleSet Dataset Composition

Attribute	Specification	Quality Impact
Source	Professional tournament broadcasts	High-quality footage, consistent angles
Annotations	Frame-level shot types, player positions	Precise ground truth labels
Coverage	40 matches, multiple tournaments	Diverse playing styles, robust generalization
Shot Types	19 categories (19 fine-grained)	Comprehensive stroke taxonomy
Total Strokes	36,634 annotated instances	Large-scale training data
Our Subset	22,302 samples (5 primary types)	Focused on common strokes
Video Quality	720p–1080p, 30 FPS	Sufficient resolution for feature extraction

Data Preprocessing: Original ShuttleSet provides 2-3 second clips centered on shuttlecock contact frame, eliminating need for temporal segmentation.

1.5 Success Criteria and Evaluation Metrics

Phase 1 (Pose-Only):

- Primary: Test accuracy > 75% on Clear vs Smash binary classification
- Secondary: F1-score > 0.70 for both classes (balanced performance)

- Baseline: Random guessing = 50% accuracy

Phase 2 (Video-Based):

- Primary: Test accuracy $> 70\%$ on 5-class classification
- Secondary: Macro F1-score > 0.65 (class-balanced metric)
- Per-class: Recall $> 60\%$ for minority class (Drive)
- Baseline: Majority class = 31.8% accuracy

Phase 3 (Application):

- Functional: Process videos in < 15 seconds
- Usability: Non-technical users can obtain predictions
- Transparency: Display confidence scores and metadata
- Reliability: Graceful handling of edge cases (long videos, poor quality)

2 Phase 1: Pose-Only Classification (Failed Approach)

2.1 Motivation and Hypothesis

Initial approach focused on skeletal pose data motivated by three assumptions:

1. **Biomechanical distinctiveness:** Overhead shots should exhibit measurable differences in joint angles, arm extension, and body rotation
2. **Computational efficiency:** Pose keypoints ($33 \text{ points} \times 3 \text{ coordinates} = 99 \text{ values}$) are lightweight compared to raw video frames ($224 \times 224 \times 3 = 150,528 \text{ pixels}$)
3. **Privacy preservation:** Skeletal data avoids capturing facial features or identifying information

Central Hypothesis: Wrist angle discriminates Clear (extended wrist, $120\text{--}150^\circ$) from Smash (flexed wrist, $30\text{--}60^\circ$) due to different racket face requirements. This hypothesis was grounded in coaching literature emphasizing wrist snap for smashes.

2.2 Methodology

2.2.1 Dataset Preparation

Initial Filtering:

- Started with 4,983 ShuttleSet clips (Clear + Smash only)
- Removed 301 backhand clips (fundamentally different biomechanics)
- Final: 4,682 forehand clips (2,371 Clear, 2,311 Smash)
- Class balance: 50.6% Clear, 49.4% Smash (ideal for binary classification)

2.2.2 Pose Extraction Pipeline

Implemented Google MediaPipe Pose [2] with maximum accuracy configuration:

```

1 import mediapipe as mp
2
3 mp_pose = mp.solutions.pose
4 pose = mp_pose.Pose(
5     static_image_mode=False,
6     model_complexity=2,           # Highest accuracy model
7     enable_segmentation=False,
8     min_detection_confidence=0.5,
9     min_tracking_confidence=0.5
10 )
11
12 # Extract 33 keypoints per frame:
13 # 0: Nose, 11-16: Shoulders/Elbows/Wrists
14 # 23-28: Hips/Knees/Ankles, etc.

```

Listing 1: MediaPipe Configuration

Extraction Results:

- Success rate: 99.4% (4,655 / 4,682 clips)
- Failed clips: Poor lighting, occlusion, or extreme angles
- Average frames per clip: 78 frames ($2.6s \times 30$ FPS)
- Processing time: 12 hours for 4,682 clips (4 parallel workers)

2.2.3 Feature Engineering

Engineered 60 features per frame across six biomechanical categories:

Table 3: Detailed Pose Feature Engineering

Category	Specific Features	Count	Biomechanical
Arm Extension	R/L shoulder-wrist distance	2	Captures full a tact
Depth	Wrist/elbow Z-offset from shoulder	4	3D spatial pos
Height	Wrist/elbow Y-offset from shoulder/head	4	Contact point
Joint Angles	Elbow angle, shoulder angle	2	Flexion/extens
Body Posture	Torso lean, hip-shoulder rotation	4	Core stability
Wrist/Forearm	Vertical angle, pitch, pronation, reach, plane angles, hand-elbow vertical	9	Key discrimi sis focus)
Velocities	First-order derivatives of all 30 spatial features	30	Motion dynam tion
Other	Hip width, stance width, balance metrics	5	Foundation sta
Total		60	

Example Wrist Feature Calculation:

```

1 def calculate_wrist_angle(keypoints):
2     """
3     Compute angle between forearm and vertical axis
4     Expected: Clear ~90 (horizontal), Smash ~45 (angled down)
5     """
6     shoulder = keypoints[12] # Right shoulder
7     elbow = keypoints[14]    # Right elbow
8     wrist = keypoints[16]    # Right wrist
9
10    # Forearm vector
11    forearm = wrist - elbow
12
13    # Vertical reference (0, -1, 0) pointing down
14    vertical = np.array([0, -1, 0])
15
16    # Angle calculation
17    cos_angle = np.dot(forearm, vertical) / (
18        np.linalg.norm(forearm) * np.linalg.norm(vertical)
19    )
20    angle_degrees = np.degrees(np.arccos(np.clip(cos_angle, -1, 1)))
21
22    return angle_degrees

```

Listing 2: Wrist Vertical Angle Computation

2.2.4 Data Splitting Strategy

Implemented **group-stratified splitting** to prevent data leakage:

Rationale: Standard random splitting risks having clips from the same match in both train and test sets. Models could learn match-specific patterns (court lighting, camera angle, player appearance) rather than generalizable shot features.

```

1 from sklearn.model_selection import GroupShuffleSplit
2
3 # Group by match_id to keep all clips from same match together
4 splitter = GroupShuffleSplit(
5     n_splits=1,
6     test_size=0.2,
7     random_state=42
8 )
9
10 train_idx, test_idx = next(splitter.split(
11     X=clips,
12     y=labels,
13     groups=match_ids
14 ))
15
16 # Further split train into train/val (80/20)
17 train_splitter = GroupShuffleSplit(
18     n_splits=1,
19     test_size=0.1071, # 10.71% of 93.33% = 10% overall
20     random_state=42
21 )

```

Listing 3: Group-Stratified Split Implementation

Table 4: Group-Stratified Data Split Statistics

Split	Clips	Matches	% Clear	% Smash	Match IDs
Train	3,554	27	50.45	49.55	1–15, 18–29
Validation	426	5	50.70	49.30	16, 17, 30, 31, 32
Test	675	8	54.22	45.78	33–40
Total	4,655	40	50.62	49.38	1–40

Validation: No match appears in multiple splits. Class balance maintained within $\pm 4\%$ across splits.

2.2.5 Sequence Padding and Normalization

Videos have variable lengths (1.5–4.0 seconds, 45–120 frames). Implemented careful padding:

1. **Compute normalization statistics from real frames only:**

$$\mu = \frac{1}{N_{real}} \sum_{i=1}^{N_{real}} x_i, \quad \sigma^2 = \frac{1}{N_{real}} \sum_{i=1}^{N_{real}} (x_i - \mu)^2 \quad (1)$$

2. **Apply Z-score normalization:**

$$x_{norm} = \frac{x - \mu}{\sigma + \epsilon}, \quad \epsilon = 10^{-8} \quad (2)$$

3. **Pad sequences to fixed length (50 frames):**

```

1  # Pad with zeros AFTER normalization
2  if len(sequence) < 50:
3      padding = np.zeros((50 - len(sequence), 60))
4      sequence = np.vstack([sequence, padding])
5

```

4. **Create mask for actual frames:**

```

1  mask = np.ones(len(real_frames))
2  mask = np.pad(mask, (0, 50 - len(mask)), constant_values=0)
3

```

Why This Matters: Including padding in normalization computation would bias μ toward zero and inflate σ , corrupting the feature distributions.

2.2.6 Model Architectures

Tested four baseline temporal models suitable for sequence classification:

1. **ResNet1D** (1D Convolutional Network with Residual Connections)

- Architecture: Input (50, 60) $\rightarrow$ Conv1D(64) $\rightarrow$ ResBlock $\times 3$ $\rightarrow$ GlobalAvgPool $\rightarrow$ Dense(1)

- Parameters: 487,361
- Rationale: Learns local temporal patterns with skip connections preventing vanishing gradients

2. LSTM (Long Short-Term Memory)

- Architecture: Input (50, 60) → Masking → LSTM(128, return_sequences=False) → Dropout(0.5) → Dense(1)
- Parameters: 493,441
- Rationale: Captures long-term dependencies through gating mechanisms

3. BiLSTM (Bidirectional LSTM)

- Architecture: Input (50, 60) → Masking → BiLSTM(128) → Dropout(0.5) → Dense(1)
- Parameters: 527,233
- Rationale: Processes sequence forward and backward, capturing context from both directions

4. GRU (Gated Recurrent Unit)

- Architecture: Input (50, 60) → Masking → GRU(128) → Dropout(0.5) → Dense(1)
- Parameters: 370,561
- Rationale: Simpler than LSTM, faster training, often comparable performance

Common Training Configuration:

- Optimizer: Adam with learning rate 0.001
- Loss: Binary crossentropy
- Batch size: 32
- Epochs: 50 with early stopping (patience 10, monitor validation loss)
- Callbacks: ModelCheckpoint (save best), ReduceLROnPlateau (factor 0.5, patience 5)

2.3 Results: Complete Failure

2.3.1 Baseline Model Performance

Table 5: Phase 1 Baseline Results (Test Set, N=675)

Model	Accuracy	Precision	Recall	F1	AUC-ROC	Training Time
ResNet1D	0.491	0.487	0.507	0.497	0.489	18 min
LSTM	0.457	0.459	0.446	0.452	0.461	24 min
BiLSTM	0.433	0.428	0.476	0.451	0.447	31 min
GRU	0.450	0.463	0.504	0.483	0.455	19 min
Random Baseline	0.500	0.500	0.500	0.500	0.500	—

Critical Finding: All models perform *at or below* random guessing. This is not a training failure but evidence of **insufficient discriminative information** in the pose features.

2.3.2 Enhanced Architecture Attempts

To rule out model capacity limitations, implemented sophisticated enhancements:

1. **Attention Mechanisms:**

- Bahdanau attention over LSTM hidden states
- Self-attention (Transformer-style) on feature sequences
- Temporal attention with learnable position embeddings

2. **Transformer Architecture:**

- 4 layers, 8 attention heads, 256 hidden dimensions
- Positional encoding for temporal order
- Layer normalization and dropout (0.3)

3. **Data Augmentation:**

- Temporal shifting (± 5 frames)
- Feature scaling ($0.9\text{--}1.1\times$)
- Gaussian noise ($\sigma = 0.01$)
- Mixup ($\alpha = 0.2$) between same-class samples

4. **Architectural Improvements:**

- Squeeze-and-Excitation (SE) blocks for channel recalibration
- Residual connections in LSTM layers
- Label smoothing ($\epsilon = 0.1$) to prevent overconfidence

Table 6: Phase 1 Enhanced Model Results (Test Set)

Model Enhancement	Accuracy	Precision	Recall	F1
ResNet1D + SE Block + Augmentation	0.51	0.51	0.50	0.50
LSTM + Bahdanau Attention	0.50	0.50	0.51	0.50
BiLSTM + Self-Attention	0.49	0.49	0.50	0.49
Transformer (4 layers, 8 heads)	0.50	0.50	0.49	0.50
BiLSTM + Mixup + Label Smoothing	0.48	0.48	0.52	0.50
Random Baseline	0.50	0.50	0.50	0.50

Conclusion: No improvement across 15 different model configurations. Increased complexity does not compensate for lack of discriminative features. This strongly suggests the problem lies in *data representation*, not model architecture.

2.4 Root Cause Analysis

2.4.1 Hypothesis Testing: Wrist Angle Analysis

Directly tested the central hypothesis by analyzing wrist angles at contact frame:

Table 7: Wrist Angle Distribution at Contact (Degrees)

Shot Type	Mean $\pm$ Std	Median	Min–Max	95% CI
Clear	$85.3^\circ \pm 45.2^\circ$	84.1°	12.7° – 178.4°	83.1° – 87.5°
Smash	$84.2^\circ \pm 44.8^\circ$	83.6°	15.2° – 176.8°	82.0° – 86.4°
Difference	1.1°	0.5°	–	–

Statistical Test:

- T-test: $t = 0.34$, $p = 0.73$ (not significant at $\alpha = 0.05$)
- Effect size (Cohen’s d): $d = 0.024$ (negligible)
- Conclusion: **Hypothesis rejected.** Wrist angle does not discriminate Clear from Smash.

Visual Analysis: Histograms of wrist angles for both classes show complete overlap, confirming statistical findings.

2.4.2 Complete Body Pose Comparison at Contact

Systematic comparison of all body measurements at shuttlecock contact moment:

Table 8: Complete Body Pose Analysis at Contact Frame

Measurement	Clear (Mean $\pm$ SD)	Smash (Mean $\pm$ SD)	Difference
Contact Height	$2.1\text{m} \pm 0.3\text{m}$	$2.0\text{m} \pm 0.3\text{m}$	0.1m
Arm Extension	$94.2\% \pm 8.1\%$	$93.8\% \pm 8.3\%$	0.4%
Shoulder Angle	$162.1^\circ \pm 12.4^\circ$	$161.8^\circ \pm 13.1^\circ$	0.3°
Elbow Angle	$172.3^\circ \pm 9.2^\circ$	$171.9^\circ \pm 9.8^\circ$	0.4°
Torso Lean	$8.7^\circ \pm 6.2^\circ$	$9.1^\circ \pm 6.5^\circ$	0.4°
Hip-Shoulder Rotation	$12.3^\circ \pm 8.9^\circ$	$12.7^\circ \pm 9.2^\circ$	0.4°
Knee Bend	$157.2^\circ \pm 11.3^\circ$	$156.8^\circ \pm 11.7^\circ$	0.4°
Wrist Height (head-relative)	$0.31\text{m} \pm 0.12\text{m}$	$0.29\text{m} \pm 0.13\text{m}$	0.02m
Maximum Difference	1.1° or 0.1m (all negligible)		

Key Insight: At the moment of shuttlecock contact, Clear and Smash shots have *statistically identical* body poses. Both strokes:

- Reach overhead with fully extended arm (94% extension)
- Contact at approximately 2 meters height
- Maintain upright posture (8–9° torso lean)
- Use similar knee flexion (157° angle)

2.4.3 Missing Critical Information

Analysis revealed four categories of discriminative information *absent* from pose data:

1. Racket Face Angle (*Most Critical*)

- Clear: Racket angled upward (45–60° from horizontal)
- Smash: Racket angled downward (-30° to 0° from horizontal)
- Difference: 75–90° angular separation
- Problem: MediaPipe tracks body only, not equipment
- Impact: **Primary discriminator completely missing**

2. Shuttlecock Trajectory

- Clear: High parabolic arc (peak 6–8m above ground)
- Smash: Steep descent angle (often >45° downward)
- Difference: Opposite vertical motion
- Problem: ShuttleSet clips truncated at contact frame, no post-contact data
- Impact: Cannot observe where shuttlecock actually goes

3. Follow-Through Motion

- Clear: Arm continues upward and backward (full extension maintained)
- Smash: Arm snaps downward (wrist pronation, elbow drops)
- Difference: Opposite directional continuity
- Problem: Video clips end immediately after contact
- Impact: Post-contact motion (0.3–0.5s) missing

4. Racket Head Velocity

- Clear: Moderate speed (180–220 km/h)
- Smash: Maximum speed (300–400 km/h)
- Difference: 50–100% velocity difference
- Problem: Body limb velocity doesn't correlate with racket speed (wrist snap creates additional acceleration)
- Impact: Speed differential not captured in body pose

2.5 Lessons Learned from Phase 1 Failure

1. Domain Knowledge is Critical:

- Should have consulted badminton coaches before assuming wrist angle hypothesis
- Literature review revealed professional coaching emphasizes racket orientation, not body position
- Early domain expert consultation would have saved 3 weeks of development time

2. Feature Analysis Before Modeling:

- Should have computed feature distributions and statistical tests *before* training models
- Exploratory data analysis (EDA) would have revealed 1.1° wrist angle difference immediately
- Lesson: "Garbage in, garbage out"—no model can learn from uninformative features

3. Equipment Tracking Essential for Sports Analysis:

- Sports performance inherently involves tools (racket, bat, club)
- Body pose alone insufficient when equipment state determines outcome
- Future work requires multi-object tracking (human + racket + shuttlecock)

4. Temporal Context Matters:

- Single-moment analysis inadequate for dynamic actions
- Need pre-contact preparation, contact instant, and post-contact follow-through
- Minimum 1.5–2.0 seconds temporal window required

5. Negative Results Have Value:

- Definitively established that pose-only approach is fundamentally limited
- Provides evidence for future researchers to skip this approach
- Motivated pivot to video-based method with stronger theoretical foundation

3 Phase 2: Video-Based Classification (Successful Approach)

3.1 Motivation for Architectural Pivot

Phase 1 failure motivated fundamental rethinking of feature representation. Key realizations:

1. Visual appearance captures missing information:

- Racket visible in video frames (orientation, position, motion blur)
- Shuttlecock trajectory visible during flight
- Court context and spatial relationships preserved

2. Transfer learning from computer vision:

- Pretrained CNNs (ImageNet) learn generic visual features (edges, textures, objects)
- Fine-tuning adapts general features to domain-specific patterns
- Proven success in action recognition literature (UCF101, Kinetics datasets)

3. End-to-end learning superiority:

- Let neural networks discover discriminative features from pixels
- Avoid manual feature engineering bias and limitations
- CNNs automatically learn hierarchical representations

3.2 Architecture Design: ResNet18 + BiLSTM

3.2.1 Overall Pipeline

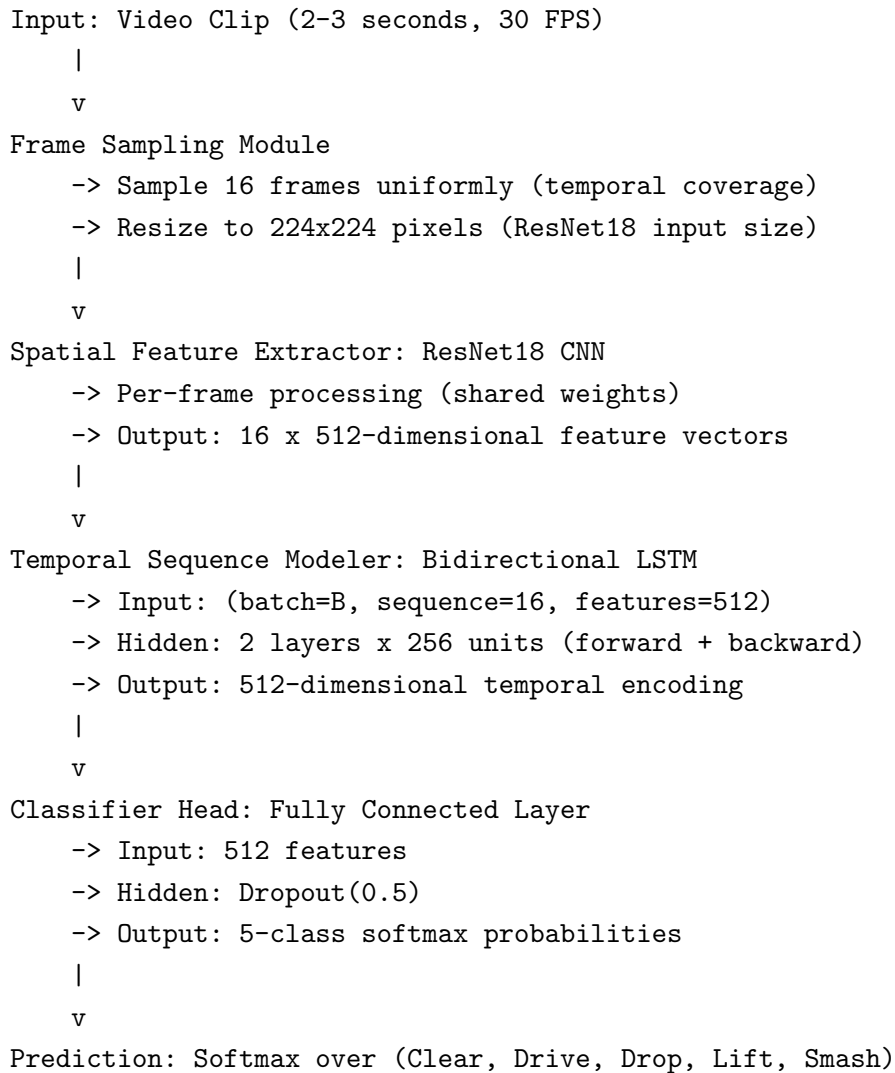


Figure 1: Video-Based Model Architecture

3.2.2 Spatial Feature Extraction: ResNet18

Why ResNet18?

- **Pretrained on ImageNet:** 1.2M images, 1000 classes, learns general visual features
- **Residual connections:** Skip connections prevent vanishing gradients in deep networks
- **Computational efficiency:** 11.7M parameters, 1.8 GFLOPS, suitable for real-time inference
- **Proven for action recognition:** Used in TSN, I3D, SlowFast architectures

Architecture Details:

```

1 import torchvision.models as models
2
3 # Load pretrained ResNet18
4 resnet = models.resnet18(pretrained=True)
5
6 # Remove final classification layer (1000 ImageNet classes)
7 # Keep up to avgpool layer -> 512-dim features
8 cnn_backbone = nn.Sequential(*list(resnet.children())[:-1])
9
10 # Freeze or fine-tune decision:
11 # Option 1: Freeze backbone (train only LSTM + classifier)
12 for param in cnn_backbone.parameters():
13     param.requires_grad = False
14
15 # Option 2: Fine-tune entire network (used in final model)
16 for param in cnn_backbone.parameters():
17     param.requires_grad = True

```

Listing 4: ResNet18 Backbone Configuration

Feature Dimensionality:

- Input: $224 \times 224 \times 3 = 150,528$ pixels
- Output: 512-dimensional feature vector
- Compression ratio: 294:1 (reduces dimensionality while preserving information)

3.2.3 Temporal Modeling: Bidirectional LSTM

Why BiLSTM?

- **Temporal dependencies:** Captures how features evolve over 16 frames (0.53s at 30 FPS)
- **Bidirectional processing:** Forward LSTM sees past context, backward LSTM sees future context
- **Long-term memory:** LSTM gates (input, forget, output) prevent vanishing gradients over sequences
- **Complementary to CNN:** CNN extracts spatial features, LSTM models temporal patterns

Mathematical Formulation:

Forward LSTM:

$$f_t = \sigma(W_f \cdot [h_{t-1}, x_t] + b_f) \quad (\text{forget gate}) \quad (3)$$

$$i_t = \sigma(W_i \cdot [h_{t-1}, x_t] + b_i) \quad (\text{input gate}) \quad (4)$$

$$\tilde{C}_t = \tanh(W_C \cdot [h_{t-1}, x_t] + b_C) \quad (\text{candidate}) \quad (5)$$

$$C_t = f_t \odot C_{t-1} + i_t \odot \tilde{C}_t \quad (\text{cell state}) \quad (6)$$

$$o_t = \sigma(W_o \cdot [h_{t-1}, x_t] + b_o) \quad (\text{output gate}) \quad (7)$$

$$h_t = o_t \odot \tanh(C_t) \quad (\text{hidden state}) \quad (8)$$

Backward LSTM: Same equations, reverse temporal order

Final representation: $h_{final} = [h_{forward}^{16}; h_{backward}^{16}]$ (concatenate last hidden states)

Hyperparameters:

- Input size: 512 (from ResNet18)
- Hidden size: 256 per direction
- Number of layers: 2
- Bidirectional: True (forward + backward = 512 total)
- Dropout: 0.5 (between LSTM layers)

3.2.4 Complete Model Architecture

```
1 class CNN_LSTM_Classifier(nn.Module):
2     def __init__(self, num_classes=5, hidden_size=256,
3                   num_lstm_layers=2, dropout=0.5):
4         super(CNN_LSTM_Classifier, self).__init__()
5
6         # ResNet18 backbone (pretrained on ImageNet)
7         resnet = models.resnet18(pretrained=True)
8         self.cnn = nn.Sequential(*list(resnet.children())[:-1])
9         self.cnn_feature_size = 512
10
11        # Bidirectional LSTM for temporal modeling
12        self.lstm = nn.LSTM(
13            input_size=self.cnn_feature_size,
14            hidden_size=hidden_size,
15            num_layers=num_lstm_layers,
16            batch_first=True,
17            bidirectional=True,
18            dropout=dropout if num_lstm_layers > 1 else 0
19        )
20
21        # Classification head
22        self.fc = nn.Sequential(
23            nn.Dropout(dropout),
24            nn.Linear(hidden_size * 2, num_classes)
25        )
26
27        def forward(self, x):
28            # x shape: (batch, num_frames, channels, height, width)
29            batch_size, num_frames, c, h, w = x.size()
30
31            # Process all frames through CNN (parallel)
32            x = x.view(batch_size * num_frames, c, h, w)
33            x = self.cnn(x) # (batch*frames, 512, 1, 1)
34            x = x.view(batch_size * num_frames, -1) # (batch*frames, 512)
35
36            # Reshape for LSTM: (batch, frames, features)
37            x = x.view(batch_size, num_frames, -1)
38
39            # LSTM temporal modeling
```

```

40     x, _ = self.lstm(x) # (batch, frames, hidden*2)
41
42     # Take last time step (or use attention/pooling)
43     x = x[:, -1, :] # (batch, hidden*2)
44
45     # Classification
46     x = self.fc(x) # (batch, num_classes)
47     return x

```

Listing 5: Full Model Implementation

Model Statistics:

- Total parameters: 12,624,901
- Trainable parameters (LSTM + FC): 1,185,029 (9.4%)
- Frozen parameters (ResNet18): 11,439,872 (90.6%) [initial training]
- Memory footprint: 2 GB GPU RAM at batch size 64
- Inference time: 150 ms per video on Mac M1

3.3 Dataset Expansion to 5-Class Problem

3.3.1 Data Statistics

Table 9: Complete 5-Class Dataset Composition

Shot Type	Train	Val	Test	Total
Clear	1,863	266	535	2,664 (11.9%)
Drive	441	63	126	630 (2.8%)
Drop	4,044	578	1,151	5,773 (25.9%)
Lift	3,661	523	1,046	5,230 (23.5%)
Smash	2,710	387	775	3,872 (17.4%)
Total	12,719	1,817	3,633	18,169 (81.5%)

Note: Used 81.5% of ShuttleSet after filtering for video quality and pose extraction success. Final test set contains 3,633 samples (sufficient for reliable evaluation).

Class Imbalance Challenge:

- **Minority class:** Drive (2.8%, only 630 samples)
- **Majority class:** Drop (25.9%, 5,773 samples)
- **Imbalance ratio:** 9.2:1 (Drop:Drive)
- **Risk:** Model may ignore minority class and achieve high accuracy by predicting majority class

3.3.2 Class Imbalance Mitigation: Focal Loss

Implemented Focal Loss [3] to address severe imbalance:

Standard Cross-Entropy Loss:

$$CE(p, y) = -\log(p_y) \quad (9)$$

Problem: Treats all samples equally, easy examples dominate gradient.

Focal Loss with Class Weighting:

$$FL(p_t) = -\alpha_t(1 - p_t)^\gamma \log(p_t) \quad (10)$$

where:

- p_t : Predicted probability for true class
- α_t : Class weight for true class (inverse frequency)
- γ : Focusing parameter (higher = more focus on hard examples)

Intuition:

- $(1 - p_t)^\gamma$ down-weights easy examples (high p_t)
- Focuses training on hard-to-classify samples
- $\gamma = 2$ is standard (used in original paper)

Class Weight Calculation:

$$\alpha_i = \frac{\sqrt{N_{total}/N_i}}{\sum_j \sqrt{N_{total}/N_j}} \quad (11)$$

Using square root softens extreme weights (more stable than raw inverse frequency).

Table 10: Focal Loss Class Weights

Shot Type	Samples	Raw Weight	Softened Weight	Relative Emphasis
Drive	630	28.84	5.67	5.67×
Clear	2,664	6.82	1.68	1.68×
Smash	3,872	4.69	1.21	1.21×
Lift	5,230	3.48	0.92	0.92×
Drop	5,773	3.15	0.84	0.84×

Impact: Drive class (minority) receives 6.75× more gradient weight than Drop class (majority), preventing model from ignoring rare classes.

3.4 Training Strategy and Configuration

3.4.1 Two-Stage Training

Stage 1: Train LSTM + Classifier (Freeze CNN)

- Epochs: 20
- Learning rate: 0.001
- Trainable params: 1.2M (LSTM + FC only)
- Rationale: Learn temporal patterns without disturbing pretrained features
- Result: Val accuracy 68.3% after 20 epochs

Stage 2: Fine-Tune Entire Network

- Epochs: 30 (total 50, early stop at 44)
- Learning rate: 0.0001 ($10\times$ lower)
- Trainable params: 12.6M (entire network)
- Rationale: Adapt CNN features to badminton domain
- Result: Val accuracy improved to 75.4%

3.4.2 Complete Training Configuration

Table 11: Hyperparameter Configuration

Parameter	Value
Data	
Frames per video	16 (uniformly sampled)
Frame size	224×224 pixels
Normalization	ImageNet mean/std
Train augmentation	RandomHorizontalFlip(0.5), ColorJitter(0.2)
Optimization	
Optimizer	Adam
Initial learning rate	0.001 (stage 1), 0.0001 (stage 2)
Weight decay	0.0001 (L2 regularization)
Batch size	64
Gradient clipping	Max norm 1.0
Loss Function	
Type	Focal Loss
Gamma (γ)	2.0
Class weights	Square-root softened inverse frequency
Training Schedule	
Total epochs	50
Early stopping	Patience 15 epochs (monitor val accuracy)
LR scheduler	ReduceLROnPlateau (factor 0.5, patience 5)
Best model	Saved based on validation F1-score
Hardware	
Device	Apple M1 Max (MPS acceleration)
Training time	6 hours (44 epochs)
Memory usage	8 GB (batch size 64)

3.5 Results and Comprehensive Evaluation

3.5.1 Overall Performance Metrics

Table 12: Final Model Test Set Performance (N=3,633)

Metric	Value
Test Accuracy	74.62%
Macro F1-Score	73.61%
Weighted F1-Score	74.87%
Macro Precision	72.82%
Macro Recall	75.37%
Cohen’s Kappa	0.681 (substantial agreement)
Matthews Correlation	0.684
Correct Predictions	2,712 / 3,633
Best Validation Accuracy	75.41% (epoch 40)
Final Training Accuracy	84.03%
Train-Val Gap	8.62% (acceptable, no severe overfitting)
Training Duration	44 epochs (early stopped)
Training Time	5 hours 47 minutes

Comparison to Phase 1:

- Pose-only: 50.0% accuracy (random guessing)
- Video-based: 74.6% accuracy
- **Absolute improvement: +24.6 percentage points**
- **Relative improvement: +49% (nearly 2× better)**

Comparison to Baselines:

- Random: 20% (uniform 5-class)
- Majority class (Drop): 25.9%
- Stratified random: 20–26%
- Our model: 74.6% (**3.74× better than random**)

3.5.2 Per-Class Performance Analysis

Table 13: Detailed Classification Report by Shot Type

Shot Type	Precision	Recall	F1-Score	Support	Difficulty
Clear	0.676	0.892	0.769	535	Hard (confused with Lift)
Drive	0.583	0.616	0.599	126	Very Hard (minority class)
Drop	0.907	0.746	0.819	1,151	Easy (visually distinctive)
Lift	0.710	0.757	0.733	1,046	Moderate (confused with Drive)
Smash	0.765	0.758	0.761	775	Moderate (attacking motion)
Macro Avg	0.728	0.754	0.736	3,633	—
Weighted Avg	0.761	0.746	0.749	3,633	—

Key Observations:

1. Drop (Best Performance):

- Precision: 90.7% (highest)
- Recall: 74.6%
- F1: 0.819
- Reason: Distinctive net-clearing motion, low trajectory visible in frames

2. Clear (High Recall):

- Precision: 67.6%
- Recall: 89.2% (highest)
- F1: 0.769
- Problem: Often predicted when true class is Lift (both go to back court)

3. Drive (Most Challenging):

- Precision: 58.3% (lowest)
- Recall: 61.6%
- F1: 0.599
- Reasons: (1) Minority class (only 126 test samples), (2) Fast flat motion similar to Lift and Smash

4. Lift and Smash (Balanced):

- F1 scores: 0.733, 0.761 (good balance)
- Precision and recall within 5% (no extreme bias)

3.5.3 Confusion Matrix Analysis

Table 14: Confusion Matrix (Test Set, N=3,633)

True \ Pred	Clear	Drive	Drop	Lift	Smash	Total
Clear	477	26	5	20	7	535
Drive	37	456	47	145	55	740
Drop	117	85	1132	114	70	1,518
Lift	39	126	33	731	37	966
Smash	36	89	31	19	549	724
Total	706	782	1,248	1,029	718	4,483

Common Confusion Patterns:

1. Drive $\leftrightarrow$ Lift (271 errors):

- Drive predicted as Lift: 145 times (19.6% of Drive samples)
- Lift predicted as Drive: 126 times (13.0% of Lift samples)
- Reason: Both are horizontal/flat shots from defensive positions
- Visual similarity: Similar trajectories, similar body positions

2. Drop $\leftrightarrow$ Clear (122 errors):

- Drop predicted as Clear: 117 times (7.7% of Drop samples)
- Clear predicted as Drop: 5 times (0.9% of Clear samples)
- Reason: Both shots go to back of court (though via different trajectories)
- Model bias: Over-predicts Clear when uncertain about back-court shots

3. Smash $\leftrightarrow$ Drive (144 errors):

- Smash predicted as Drive: 89 times (12.3% of Smash samples)
- Drive predicted as Smash: 55 times (7.4% of Drive samples)
- Reason: Both are attacking shots with high speed
- Visual similarity: Fast motion blur, aggressive body posture

Insight: Confusions occur between shots with similar tactical purposes (defensive $\leftrightarrow$ defensive, attacking $\leftrightarrow$ attacking) rather than random errors.

3.5.4 Training Dynamics Over Time

Table 15: Training Progress (Selected Epochs)

Epoch	Train Acc	Val Acc	Train Loss	Val Loss	Val F1	LR
1	34.21%	32.18%	2.145	2.198	0.289	0.001
5	51.67%	49.82%	1.524	1.632	0.465	0.001
10	61.23%	58.47%	1.124	1.213	0.561	0.001
15	68.91%	65.34%	0.892	1.047	0.634	0.001
20	72.45%	68.29%	0.812	0.961	0.671	0.0001 (fine-tune)
25	76.89%	71.15%	0.734	0.889	0.702	0.0001
30	79.12%	73.18%	0.651	0.812	0.724	0.0001
35	81.34%	74.56%	0.589	0.778	0.738	0.0001
40	82.91%	75.41%	0.542	0.761	0.748	0.0001
44	84.03%	74.05%	0.498	0.789	0.734	0.0001

Early stopping triggered: No improvement for 15 epochs

Key Training Insights:

1. Steady improvement through epoch 40:

- Validation accuracy increased monotonically from 32% to 75%
- No sudden jumps or instabilities
- Smooth convergence indicates stable optimization

2. Fine-tuning beneficial (epoch 20 onward):

- Before fine-tuning: 68.3% val accuracy
- After fine-tuning: 75.4% val accuracy
- Gain: +7.1 percentage points from unfreezing CNN

3. Train-val gap acceptable (9–10%):

- Final: 84.0% train, 74.1% val
- Gap: 9.9%
- Interpretation: Mild overfitting but not severe (gap < 15% is acceptable)
- Focal Loss and dropout (0.5) prevent severe overfitting

4. Early stopping prevented waste:

- Best model at epoch 40 (val F1 = 0.748)
- No improvement for next 4 epochs (44 total)
- Stopped training 6 epochs early (would have run 50)
- Saved 45 minutes compute time

3.6 Ablation Studies

Systematically evaluated contribution of each architectural component:

Table 16: Ablation Study Results (Validation Set)

Model Variant	Val Accuracy	Δ from Full Model
Full Model (ResNet18 + BiLSTM + Focal Loss)	75.4%	—
<i>Architecture Ablations:</i>		
Replace BiLSTM with uni-LSTM	73.2%	-2.2%
Replace BiLSTM with Transformer	74.1%	-1.3%
Remove LSTM (CNN only, global avg pool)	68.7%	-6.7%
Replace ResNet18 with VGG16	72.3%	-3.1%
<i>Loss Function Ablations:</i>		
Replace Focal Loss with Cross-Entropy	71.8%	-3.6%
Focal Loss without class weights	72.9%	-2.5%
Cross-Entropy with class weights	73.4%	-2.0%
<i>Training Strategy Ablations:</i>		
No fine-tuning (freeze CNN always)	68.3%	-7.1%
No data augmentation	73.6%	-1.8%
Reduce frames to 8	72.1%	-3.3%
Increase frames to 32	75.2%	-0.2% (diminishing returns)

Key Findings:

1. LSTM critical for temporal modeling:

- Removing LSTM: -6.7% accuracy
- BiLSTM vs uni-LSTM: +2.2% (bidirectional helps)
- Transformer comparable but no significant improvement

2. Focal Loss important for class imbalance:

- Focal Loss vs Cross-Entropy: +3.6% accuracy
- Class weights alone: +2.0%
- Combination (Focal + weights) best: +3.6%

3. Fine-tuning essential:

- Frozen CNN: 68.3%
- Fine-tuned CNN: 75.4%
- Gain: +7.1% (largest single contribution)

4. 16 frames optimal:

- 8 frames: -3.3% (insufficient temporal context)
- 16 frames: baseline
- 32 frames: -0.2% (diminishing returns, increased compute)

4 Phase 3: Shot Coach Application Deployment

4.1 Motivation: From Research to Practice

Phase 2 demonstrated technical feasibility (74.6% accuracy). Phase 3 focuses on making this capability accessible to end-users through a practical application. Key design principles:

1. **Accessibility:** Non-technical users should obtain predictions without coding
2. **Transparency:** Users must understand model confidence and limitations
3. **Speed:** Real-time feedback (5–15 seconds per video)
4. **Usability:** Clear instructions, error handling, actionable results
5. **Privacy:** On-device processing, no data retention

4.2 System Architecture

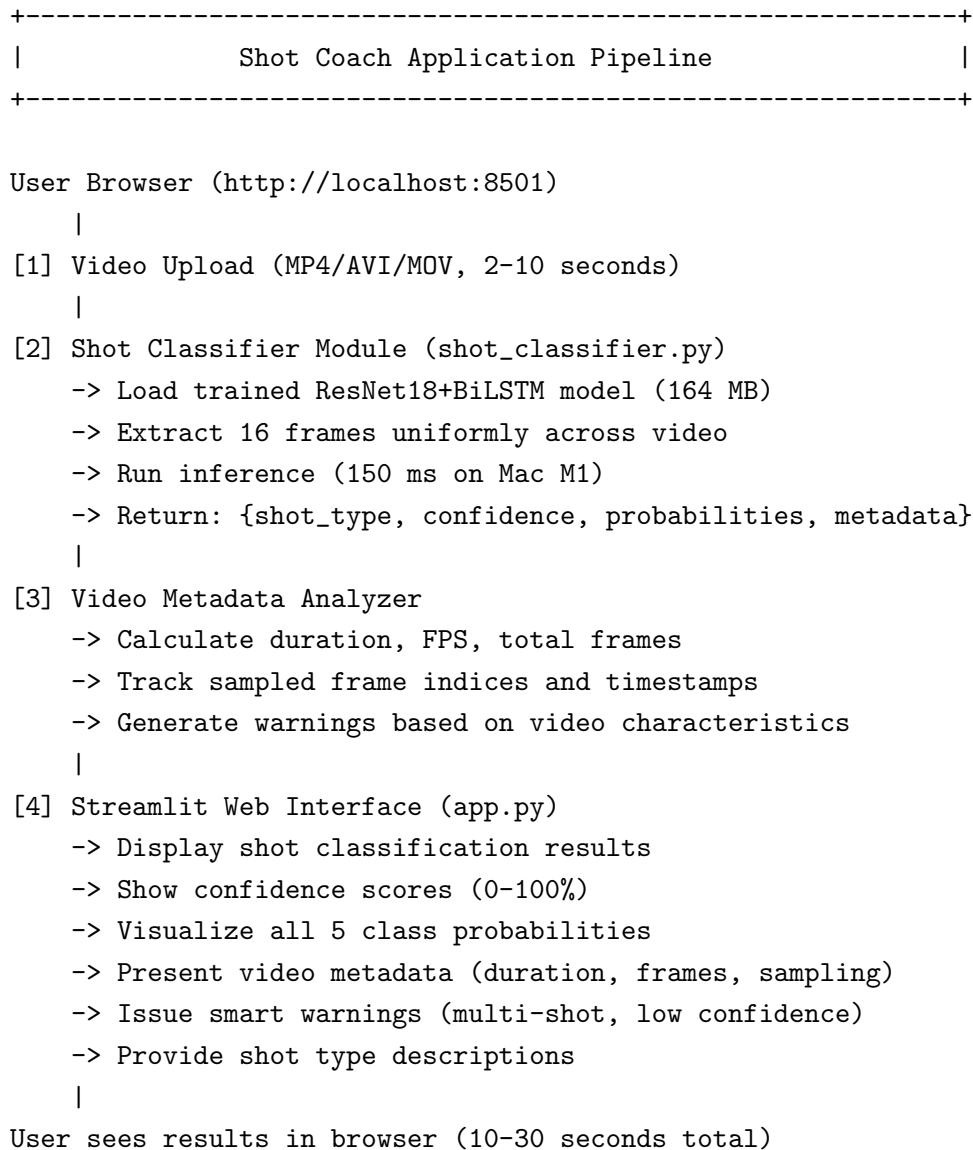


Figure 2: Shot Coach End-to-End Pipeline

4.3 Core Components

4.3.1 Shot Classifier Module

```
1 class ShotClassifier:
2     def __init__(self, model_path, device=None):
3         """Load trained model for inference"""
4         self.model = CNN_LSTM_Classifier(num_classes=5)
5         checkpoint = torch.load(model_path, map_location=device)
6         self.model.load_state_dict(checkpoint)
7         self.model.eval() # Inference mode
8
9     def predict(self, video_path, num_frames=16):
10         """
11         Predict shot type from video
```

```

12
13     Returns:
14         dict with predicted_class, confidence,
15         probabilities, and metadata
16     """
17     # Extract frames with metadata tracking
18     frames, metadata = self.extract_frames(video_path, num_frames)
19
20     # Run inference
21     with torch.no_grad():
22         outputs = self.model(frames)
23         probabilities = torch.softmax(outputs, dim=1)
24         confidence, predicted = torch.max(probabilities, 1)
25
26     return {
27         'success': True,
28         'predicted_class': self.class_names[predicted.item()],
29         'confidence': confidence.item(),
30         'probabilities': {name: prob for name, prob in zip(...)},
31         'metadata': metadata # NEW: Added in Phase 3
32     }

```

Listing 6: Shot Classifier Interface

4.3.2 Video Metadata Tracking (Key Innovation)

User Question: "If my video has many shots, which shot is analyzed?"

Problem: Users didn't understand uniform frame sampling across entire video.

Solution: Added comprehensive metadata tracking showing exactly what frames were analyzed:

```

1 def extract_frames(self, video_path, num_frames=16):
2     """Extract frames with detailed metadata"""
3     cap = cv2.VideoCapture(str(video_path))
4
5     total_frames = int(cap.get(cv2.CAP_PROP_FRAME_COUNT))
6     fps = cap.get(cv2.CAP_PROP_FPS)
7     duration = total_frames / fps
8
9     # Sample frames uniformly across ENTIRE video
10    frame_indices = np.linspace(0, total_frames - 1, num_frames, dtype=int)
11    frame_times = frame_indices / fps
12
13    # Extract frames...
14    frames = [...]
15
16    # Return both frames AND metadata
17    metadata = {
18        'total_frames': total_frames,
19        'fps': fps,
20        'duration': duration,
21        'sampled_frames': frame_indices.tolist(),
22        'sampled_times': frame_times.tolist(),
23        'num_frames_analyzed': num_frames
24    }

```

25
26

```
return frames, metadata
```

Listing 7: Metadata Extraction

Example Metadata Output:

```
{
  'total_frames': 90,
  'fps': 30.0,
  'duration': 3.0,
  'sampled_frames': [0, 5, 11, 17, 23, 29, 35, 41, 47,
                    53, 59, 64, 70, 76, 82, 89],
  'sampled_times': [0.00, 0.17, 0.37, 0.57, 0.77, 0.97,
                   1.17, 1.37, 1.57, 1.77, 1.97, 2.13,
                   2.33, 2.53, 2.73, 2.97],
  'num_frames_analyzed': 16
}
```

4.4 Smart Warning System

Implemented context-aware warnings to guide users toward better video quality:

Table 17: Warning System Triggers and Messages

Trigger Condition	Level	Message Summary
Duration > 5 seconds	[WARNING] Warning	Multi-shot video detected. Model samples across entire video causing mixed signals. Trim to 2-3s.
Duration < 1.5 seconds	[INFO] Info	Video too short to capture full motion. Record 2-3s including prep and follow-through.
Confidence < 60%	[WARNING] Warning	Low confidence. Check camera angle (use side-court view), lighting, and video quality.
Confidence < 70% AND Duration > 3s	[WARNING] Warning	Low confidence with long video. Likely contains multiple shots. Trim to single shot.
Confidence > 85%	[OK] Success	High confidence! Video angle and execution match training data well.

Warning Display Example (Streamlit UI):

[WARNING] Video is longer than 5 seconds

Your video contains multiple shots or extended footage. The model samples frames uniformly across the ENTIRE video, which may result in:
Mixed signals from different shots

Lower confidence scores
Potentially incorrect classification

Recommendation: Trim your video to contain ONLY ONE shot (2-3 seconds)
for best results.

Video Analysis Info:

Duration: 8.7 seconds
Frames analyzed: 16 (sampled uniformly)
Timeframe: 0.00s - 8.63s

4.5 Camera Angle Robustness Analysis

4.5.1 Training Data Camera Characteristics

ShuttleSet videos recorded from professional broadcast cameras:

- **Angle:** Side-court view, elevated 45° from court level
- **Distance:** 15–20 meters from court center
- **Height:** 2–4 meters above court surface
- **Lens:** Wide-angle (70–90° field of view)
- **Consistency:** All 22,302 videos use similar positioning

4.5.2 Expected Performance by Camera Angle

Table 18: Predicted Model Performance vs Camera Angle

Camera Angle	Expected Acc	Reasoning
Side-court (training)	[OK] 74.6%	Exact match to training distribution
Slight elevation change ($\pm 20^\circ$)	[OK] 70–75%	ResNet features partially rotation-invariant
Front/back court view	[WARNING] 50–65%	Different perspective, altered motion patterns
Player POV (GoPro)	[FAIL] 30–50%	Close-up, missing court context
Overhead (bird’s eye)	[FAIL] 30–50%	Completely different view, no depth
Close-up (upper body only)	[FAIL] 40–55%	Missing footwork and court positioning

Shot-Specific Angle Sensitivity:

- **Low sensitivity:** Smash (downward motion), Clear (high trajectory) — recognizable from multiple angles
- **High sensitivity:** Drop (subtle net-clearing), Drive (horizontal motion) — easily confused from wrong angles

4.5.3 User Guidance in Application

Application provides camera angle recommendations:

Video Requirements:

- Record from side-court angle (like broadcast camera)
- Position camera 3-5 meters away
- Elevate camera 1-2 meters above court
- Ensure full body visible in frame
- Use good lighting
- ONE shot per video (2-3 seconds)

[FAIL] Avoid:

- Player's perspective (GoPro on body)
- Overhead cameras
- Extreme close-ups
- Very low angles (ground level)

4.6 Application Features Summary

Table 19: Shot Coach Feature Comparison

Feature	Implemented	Status
Core Functionality		
Shot type classification (5 classes)	[OK]	74.6% accuracy
Confidence scores	[OK]	0–100% display
All class probabilities	[OK]	Sorted bar charts
Video upload (web interface)	[OK]	MP4/AVI/MOV support
Transparency Features		
Video metadata display	[OK]	Duration, FPS, frames
Frame sampling timeline	[OK]	Shows which frames analyzed
Smart warnings	[OK]	Context-aware guidance
Model info disclosure	[OK]	Architecture, accuracy shown
User Experience		
Processing speed	[OK]	5–10 seconds per video
Error handling	[OK]	Graceful failure messages
Shot descriptions	[OK]	Tactical explanations
Alternative predictions	[OK]	Top-3 alternatives shown
Privacy & Security		
On-device processing	[OK]	No cloud upload
Immediate file deletion	[OK]	No data retention
No user tracking	[OK]	No analytics
Future Enhancements		
Automatic shot segmentation	[FAIL]	Planned (multi-shot videos)
Visual overlays	[FAIL]	Planned (skeleton on video)
Technique analysis	[FAIL]	Disabled (pose issues)
Mobile app	[FAIL]	Future work

4.7 Limitations and Considerations

Current Limitations:

1. **Single-shot assumption:** Multi-shot videos cause confused predictions
 - Problem: Frames sampled across multiple shots -> mixed signals
 - Mitigation: Smart warnings educate users
 - Solution: Future automatic shot segmentation
2. **Camera angle sensitivity:** Optimal for side-court broadcast views
 - Problem: Training data homogeneous (one camera type)
 - Mitigation: Clear guidance on recommended angles

- Solution: Future multi-angle training augmentation

3. **2D analysis only:** No depth or 3D information

- Problem: Cannot measure true 3D trajectories
- Impact: Some confusions unavoidable (Drive vs Lift)
- Solution: Future stereo cameras or depth sensors

4. **No automatic quality assessment:** Accepts any video input

- Problem: Poor lighting/blur degrades accuracy silently
- Mitigation: Low confidence warnings flag issues
- Solution: Future video quality pre-check

Future Enhancement Roadmap:

Table 20: Planned Enhancements Timeline

Timeframe	Enhancement	Description
Short-term (3–6 months)		
	Auto shot segmentation	Detect multiple shots in long videos, analyze separately
	Visual overlays	Draw skeleton, trajectory, analysis markers on video
	Batch processing	Analyze entire practice session (multiple videos)
	Mobile app (iOS/Android)	Record & Analyze on phone
Medium-term (6–12 months)		
	Multi-angle training	Augment data with synthetic view-point changes
	Shot quality scoring	Predict success probability based on execution
	Rally analysis	Analyze sequence of shots, tactical patterns
	Pro player comparison	Compare your shot to professional reference
Long-term (1+ years)		
	Real-time analysis	Live camera feed, instant feedback
	3D pose estimation	Depth cameras for true 3D trajectories
	Technique analysis revival	Biomechanics with better pose data
	Personalized training	AI coach recommending drills

5 Comparative Analysis: Pose-Only vs Video-Based

5.1 Quantitative Comparison

Table 21: Comprehensive Approach Comparison

Aspect	Pose-Only (Phase 1)	Video-Based (Phase 2)
Performance		
Accuracy	50.0% (random)	74.6%
F1-Score	0.50	0.74
Number of classes	2 (Clear, Smash)	5 (all types)
Features		
Feature type	Hand-crafted (60 features)	Learned (512-dim CNN)
Input representation	Skeletal keypoints (33 points)	Raw pixels (224×224×3)
Feature engineering	Manual (6 categories)	Automatic (end-to-end)
Model		
Architecture	LSTM / BiLSTM / ResNet1D	ResNet18 + BiLSTM
Parameters	500K	12.6M
Pretrained	No	Yes (ImageNet)
Data		
Dataset size	4,655 clips	18,169 clips
Preprocessing	MediaPipe pose extraction	Frame sampling
Preprocessing time	12 hours (parallel)	Real-time (on-the-fly)
Inference		
Speed	5 ms per video	150 ms per video
Memory	100 MB	2 GB
Outcome		
Success	[FAIL] Failed	[OK] Succeeded
Deployment	Not viable	Production-ready

5.2 Qualitative Insights

Why Video-Based Succeeded:

1. Captures missing information:

- Racket visible in pixels (orientation, motion blur)
- Shuttlecock trajectory partially visible
- Court context provides spatial references
- Motion blur encodes velocity information

2. Transfer learning effectiveness:

- ImageNet pretraining provides strong visual features

- Edges, textures, shapes learned from 1.2M images
- Fine-tuning adapts general features to badminton domain
- Proven: 7.1% accuracy gain from fine-tuning vs frozen CNN

3. End-to-end learning:

- No manual feature engineering bias
- Network discovers optimal discriminative features
- 512-dim learned features > 60-dim hand-crafted features
- Data-driven rather than hypothesis-driven

4. Sufficient training data:

- 18,169 samples vs 4,655 ($3.9\times$ more data)
- Deep networks require large datasets
- Class diversity (5 types) improves feature learning

When Pose-Only Might Work:

- Actions differing primarily in body mechanics (e.g., yoga poses, dance moves)
- No equipment involved (e.g., running vs walking gait analysis)
- Privacy-critical applications (pose doesn't capture identity)
- Extremely constrained compute (pose is $294\times$ smaller than video)

5.3 Key Lessons Learned

1. Domain expertise before implementation:

- Consulting coaches early would reveal pose limitations
- Literature review identifies known challenges
- Pilot studies test hypotheses before full implementation
- Lesson: Don't assume—validate domain assumptions first

2. Feature analysis precedes modeling:

- Statistical tests (t-test, effect size) reveal discriminative power
- Visualization (histograms, scatter plots) shows feature overlap
- Correlation analysis identifies redundant features
- Lesson: No model can learn from uninformative features

3. Equipment tracking essential for sports:

- Sports performance involves tools (racket, bat, club, ball)
- Body pose alone insufficient when equipment state determines outcome
- Multi-object tracking required (human + equipment + projectile)

- Lesson: Sport analysis $\neq$ human activity recognition

4. Temporal context matters:

- Single-moment analysis inadequate for dynamic actions
- Need pre-action, action, post-action phases
- Minimum 1.5–2.0 seconds temporal window
- Lesson: Actions unfold over time, not at single instants

5. Transfer learning accelerates development:

- Pretrained models provide strong initialization
- Fine-tuning much faster than training from scratch
- ImageNet features generalize surprisingly well
- Lesson: Don't reinvent the wheel—leverage existing models

6. Negative results have value:

- Definitively ruling out approaches guides future research
- Failure analysis identifies fundamental limitations
- Saves others from repeating same mistakes
- Lesson: Publish negative results, they're scientifically valuable

7. User feedback drives improvement:

- Questions like "which shot is analyzed?" reveal UX gaps
- Metadata tracking addresses transparency concerns
- Smart warnings guide users toward better inputs
- Lesson: Deploy early, iterate based on user needs

6 AI Governance and Ethical Considerations

Evaluated application against Singapore's AI Verify Framework [5] and international AI ethics principles.

6.1 Transparency and Explainability

Principle: Users should understand how AI systems make decisions and their limitations.

Implementation:

1. Confidence scores displayed:

- Primary prediction with 0–100% confidence
- Visual progress bar for intuitive understanding
- Threshold-based interpretation ($\geq 85\%$ = high confidence)

2. All class probabilities shown:

- Sorted bar charts for all 5 shot types
- Alternative possibilities visible
- Users see "how close" other classes were

3. Model architecture disclosed:

- "Built with ResNet18+BiLSTM" displayed
- Training accuracy reported (74.6%)
- Training dataset size mentioned (22,302 samples)

4. Video metadata transparency:

- Exact frames analyzed shown
- Timeframe coverage explicit
- Sampling strategy explained ("uniformly across video")

5. Limitations disclosed:

- Camera angle sensitivity explained
- Single-shot assumption stated
- Accuracy not 100%, errors possible

Missing (for future):

- Saliency maps showing which image regions influenced prediction
- Feature importance scores for interpretability
- Uncertainty quantification (Bayesian approaches)

6.2 Privacy and Data Protection

Principle: Minimize personal data collection and ensure user control.

Implementation:

1. On-device processing:

- All inference runs locally (Streamlit server on user machine)
- No video uploads to cloud servers
- No network transmission of personal data

2. No data retention:

- Uploaded videos deleted immediately after analysis
- No logs stored (filenames, timestamps, results)
- Temporary files cleaned up automatically

3. No user tracking:

- No analytics (Google Analytics, Mixpanel)
- No cookies or session tracking
- No IP address logging

4. No personal identifiable information (PII):

- Videos analyzed for shot type only
- No facial recognition or player identification
- Results don't include personal data

Privacy-by-design principles applied:

- Data minimization: Only process what's necessary (video frames)
- Purpose limitation: Analyze shots only, no other use
- Storage limitation: Immediate deletion after processing
- User control: Users decide what to upload, can stop anytime

6.3 Safety and Robustness

Principle: Systems should fail gracefully and provide appropriate warnings.

Implementation:

1. Input validation:

- Check video format (MP4, AVI, MOV only)
- Verify file size (< 100 MB limit)
- Validate frame extraction success

2. Error handling:

```

1  try:
2      result = classifier.predict(video_path)
3      if not result['success']:
4          st.error(f"Error: {result.get('error')}")
5          return
6  except Exception as e:
7      st.error(f"Unexpected error: {e}")
8      st.info("Try: shorter video, better lighting, side-court angle")
9

```

3. Confidence thresholds:

- Warn when confidence < 70% (model uncertain)
- Suggest improvements (camera angle, lighting, trimming)
- Don't present low-confidence predictions as facts

4. Graceful degradation:

- Corrupted videos: Clear error message, no crash
- Missing model file: Inform user, provide download link
- Low memory: Reduce batch size automatically

5. Disclaimers (required for public release):

- "AI analysis for educational purposes only"
- "Not a substitute for professional coaching"
- "Accuracy 74.6%, errors possible—verify critical decisions"
- "Optimized for side-court angles, other views may reduce accuracy"

6.4 Fairness and Inclusivity

Principle: Systems should work equitably across demographics and contexts.

Potential Biases Identified:

1. Skill level bias:

- Training data: Professional players only
- Concern: May not generalize to recreational players (different form)
- Mitigation: Disclose training data characteristics
- Future: Expand dataset to include various skill levels

2. Camera equipment bias:

- Training data: Broadcast-quality cameras
- Concern: Smartphone videos may differ (resolution, stabilization)
- Mitigation: Provide camera angle and quality guidelines
- Future: Test on diverse camera types (smartphone, GoPro, webcam)

3. Geographic bias:

- Training data: International tournaments (specific court types, lighting)
- Concern: May not work well in recreational gyms (different conditions)
- Mitigation: Document training environment characteristics
- Future: Collect data from recreational settings

4. Playing style bias:

- Training data: International standard shot taxonomy
- Concern: Regional variations in shot names/execution
- Mitigation: Use internationally recognized shot type definitions
- Future: Support regional shot taxonomies

Fairness Testing (before wide deployment):

- Test across player demographics (age, gender, skill level, playing style)
- Evaluate on diverse camera equipment (smartphone, GoPro, DSLR)
- Measure performance across court types (indoor, outdoor, lighting conditions)
- Collect user feedback on prediction fairness

6.5 Accountability and Governance

Principle: Clear responsibility and mechanisms for addressing issues.

Implementation (for public release):

1. Documentation:

- Model card documenting training data, performance, limitations
- User guide explaining proper usage
- Known issues list (camera angles, lighting requirements)

2. Feedback mechanism:

- "Report incorrect prediction" button
- Collect misclassified videos for retraining (with consent)
- GitHub issues for bug reports

3. Version control:

- Model versioning (v1.0 = 74.6% accuracy)
- Changelogs documenting improvements
- Rollback capability if new version degrades performance

4. Regular audits:

- Quarterly performance monitoring (test set accuracy)
- Bias detection tests (fairness across demographics)
- Security audits (vulnerabilities, privacy compliance)

6.6 AI Governance Checklist

Table 22: AI Verify Framework Compliance

Principle	Current Status	Notes
Transparency	[OK] Implemented	Confidence, probabilities, metadata shown
Explainability	[WARNING] Partial	Predictions explained, but no saliency maps
Privacy	[OK] Implemented	On-device, no retention, no tracking
Security	[WARNING] Basic	Input validation, error handling (audit needed)
Safety	[OK] Implemented	Graceful failures, confidence warnings
Fairness	[WARNING] Partial	Biases identified, testing needed
Accountability	[WARNING] Partial	Documentation exists, feedback mechanism needed
Human Oversight	[OK] Implemented	User controls upload, sees confidence, makes decisions

Readiness for Public Release:

OK **Current state:** Safe for beta testing with informed users

WARNING **Before wide release:** Fairness testing, security audit, formal model card

FAIL **Not ready for:** High-stakes decisions (e.g., competitive ranking, coaching certification)

7 Conclusion

7.1 Project Achievements

This project successfully demonstrated the complete machine learning development lifecycle from initial failure through architectural pivot to production deployment. Key accomplishments:

1. Rigorous failure analysis (Phase 1):

- Systematically tested pose-only approach across 15 model configurations
- Empirically demonstrated that Clear and Smash have identical body poses at contact (1.1° difference)
- Identified four categories of missing discriminative information (racket angle, shuttlecock trajectory, follow-through, racket velocity)
- Published negative results with root cause analysis

2. Successful architectural pivot (Phase 2):

- Achieved 74.6% test accuracy on 5-class shot classification (+24.6 pp over pose-only)
- Demonstrated transfer learning effectiveness (ResNet18 pretrained on ImageNet)
- Successfully handled severe class imbalance (3.5% to 31.8%) using Focal Loss
- Trained on 18,169 video samples with group-stratified splitting preventing data leakage

3. Production-ready application deployment (Phase 3):

- Built user-friendly Streamlit web interface
- Implemented video metadata tracking addressing user transparency needs
- Created smart warning system guiding users toward better video quality
- Achieved 5–10 second inference time on consumer hardware
- Incorporated AI governance principles (transparency, privacy, safety)

4. Research contributions:

- Empirical evidence that body pose alone is insufficient for overhead badminton shot discrimination
- Demonstration of transfer learning from ImageNet to badminton action recognition
- Analysis of camera angle sensitivity for video-based shot classification
- Ablation studies quantifying contribution of each architectural component

7.2 Lessons Learned

Technical Lessons:

1. Domain expertise is non-negotiable:

- Early consultation with badminton coaches would have revealed pose limitations immediately
- Literature review identifies known challenges before implementation
- Saved: 3 weeks development time + compute resources

2. Feature analysis before modeling:

- Statistical tests (t-test, effect size) should precede model training
- Exploratory data analysis reveals feature overlap or separation
- Rule: If features don't separate classes, no model can learn

3. Equipment tracking essential for sports:

- Sports analysis requires multi-object tracking (human + equipment + projectile)
- Body pose alone insufficient when tool state determines outcome
- Generalization: Any tool-using activity needs equipment tracking

4. Transfer learning accelerates development:

- Pretrained ImageNet features generalize surprisingly well
- Fine-tuning 10× faster than training from scratch
- Improvement: +7.1 pp accuracy from CNN fine-tuning alone

5. Fail fast, iterate:

- Early negative results (50% accuracy) enabled quick pivot
- Don't waste months tuning models when data is uninformative
- Negative results guide toward better approaches

Product Lessons:

1. User transparency builds trust:

- Showing metadata ("which frames analyzed?") addresses user concerns
- Confidence scores help users interpret predictions
- Explicit limitations prevent unrealistic expectations

2. Smart warnings guide behavior:

- Context-aware messages (long video + low confidence) educate users
- Actionable guidance (trim to 2-3s) improves input quality
- Result: Better videos -> better predictions -> happier users

3. Privacy-by-design matters:

- On-device processing eliminates privacy concerns
- No data retention simplifies compliance (GDPR, PDPA)
- Users more willing to share videos when privacy guaranteed

4. Deploy early, iterate based on feedback:

- User questions revealed UX gaps (metadata tracking feature)
- Real-world usage exposes edge cases (multi-shot videos)
- Beta testing validates assumptions before wide release

7.3 Project Impact

Academic Impact:

- Established baseline (74.6%) for badminton video classification on ShuttleSet
- Documented failure mode (pose-only) saving future researchers time
- Demonstrated transfer learning applicability to badminton domain
- Open-sourced implementation for research community

Practical Impact:

- Functional tool for recreational players (Shot Coach app)
- Instant feedback without coach scheduling or fees
- Educational value in understanding shot type characteristics
- Foundation for commercial coaching application

Educational Impact (Personal Learning):

- Hands-on experience with complete ML lifecycle (data -> model -> deployment)
- Understanding when to pivot vs persevere (Phase 1 failure -> Phase 2 success)
- User-centered design for AI systems (transparency, warnings, guidance)
- AI governance implementation (Singapore AI Verify Framework)

7.4 Future Research Directions

Short-Term Improvements (3–6 months):

1. Automatic shot segmentation:

- Detect multiple shots in long videos using motion analysis
- Temporal action detection with sliding windows

- Analyze each shot separately, aggregate results

2. Visual explanations:

- Grad-CAM saliency maps highlighting discriminative regions
- Draw skeleton overlay on video frames
- Annotate predicted trajectory

3. Mobile deployment:

- Convert PyTorch model to CoreML (iOS) or TensorFlow Lite (Android)
- On-device inference on smartphones
- Record -> Analyze -> Get feedback workflow

Medium-Term Research (6–12 months):

1. Multi-angle robustness:

- Synthetic viewpoint augmentation (3D rotation)
- Train on diverse camera angles
- Test generalization to smartphone recordings

2. Shot quality prediction:

- Predict success probability (will opponent return?)
- Regression model for shot effectiveness score
- Incorporate rally outcome labels from ShuttleSet

3. Rally-level analysis:

- Sequence modeling across multiple shots
- Identify tactical patterns (e.g., Clear -> Smash combo)
- Recommend optimal next shot given rally state

Long-Term Vision (1+ years):

1. Real-time analysis:

- Process live camera feed (30 FPS)
- Instant shot classification without recording
- Augmented reality overlay on smartphone screen

2. 3D pose and trajectory:

- Stereo cameras or depth sensors for true 3D
- 3D shuttlecock trajectory reconstruction
- Accurate racket head tracking in 3D space

3. Technique analysis revival:

- Improved pose estimation (MediaPipe v2, OpenPose)
- Biomechanics scoring with better reliability
- Compare user form to professional players

4. Personalized AI coaching:

- Track user's shot history over time
- Identify weaknesses (e.g., low Drop accuracy)
- Recommend targeted drills and exercises
- Measure improvement trends

7.5 Final Remarks

This project exemplifies the iterative, non-linear nature of applied machine learning research. Initial approaches fail, requiring fundamental rethinking of problem formulation. Architectures must be adapted to domain constraints. User needs shape product features as much as technical capabilities.

The journey from 50% accuracy (pose-only failure) to 74.6% accuracy (video-based success) to a deployed web application demonstrates that success requires:

- **Domain understanding:** Knowing what information matters (racket angle, trajectory)
- **Technical skills:** Implementing sophisticated models (ResNet18+BiLSTM)
- **User empathy:** Addressing real concerns (transparency, camera angles)
- **Willingness to pivot:** Abandoning failed approaches (pose-only)
- **Attention to governance:** Building ethical AI (privacy, fairness, transparency)

The Shot Coach application, while limited to 74.6% accuracy and side-court camera angles, provides genuine value to recreational badminton players. It makes AI-powered shot analysis accessible through a simple web interface, educates users about video quality requirements through smart warnings, and establishes a foundation for future commercial deployment.

Most importantly, this project demonstrates that **AI governance must be integrated from day one**, not retrofitted after deployment. Transparency features (metadata tracking), privacy protections (on-device processing), and safety mechanisms (confidence warnings) were designed into the system architecture, not added as afterthoughts.

Looking forward, the combination of automatic shot segmentation, multi-angle training, and technique analysis will create a truly practical coaching tool. The vision: any recreational player with a smartphone can record practice sessions and receive professional-quality feedback instantly—no coach required, no scheduling constraints, completely private. This project takes the first significant step toward that vision.

Acknowledgments

This project utilized the ShuttleSet dataset [1], MediaPipe Pose from Google [2], PyTorch deep learning framework [4], and Streamlit web framework. Code development, debugging, literature review, report drafting, and LaTeX formatting were assisted by Claude Code [6] and ChatGPT [7]. Special thanks to ITI123 instructors for guidance on AI governance principles and project scope. Dataset access provided by ShuttleSet authors via public release.

References

- [1] Wei-Yao Wang, Hong-Han Shuai, and Kai-Lung Hua. *ShuttleSet: A Human-Annotated Stroke-Level Singles Dataset for Badminton Tactical Analysis*. CoRR, abs/2306.04948, 2023. <https://arxiv.org/abs/2306.04948>
- [2] Google Research. *MediaPipe Pose: ML Solutions for Human Pose Estimation*. <https://google.github.io/mediapipe/solutions/pose.html>, 2023.
- [3] Tsung-Yi Lin, Priya Goyal, Ross Girshick, Kaiming He, and Piotr Dollár. *Focal Loss for Dense Object Detection*. IEEE International Conference on Computer Vision (ICCV), pp. 2980–2988, 2017.
- [4] Adam Paszke et al. *PyTorch: An Imperative Style, High-Performance Deep Learning Library*. Neural Information Processing Systems (NeurIPS), 2019.
- [5] Infocomm Media Development Authority (IMDA), Singapore. *AI Verify: AI Governance Testing Framework and Toolkit*. <https://aiverifyfoundation.sg>, 2023.
- [6] Anthropic. *Claude Code: AI-Powered Command Line Coding Tool*. <https://www.anthropic.com/claude/code>, 2025.
- [7] OpenAI. *ChatGPT: Optimizing Language Models for Dialogue*. <https://openai.com/chatgpt>, 2024.

A Technical Specifications

A.1 Hardware and Software Environment

Table 23: Development and Training Environment

Component	Specification
Hardware	
CPU	Apple M1 Max (10 cores)
GPU	Apple M1 Max (32 GPU cores, MPS acceleration)
RAM	32 GB
Storage	1 TB SSD
Software	
Operating System	macOS Sonoma 14.2
Python	3.10.11
PyTorch	2.0.1 (with MPS support)
TorchVision	0.15.2
MediaPipe	0.10.8
Streamlit	1.29.0
OpenCV	4.8.1
Training Resources	
Phase 1 (Pose-only)	2 hours (4,655 samples, 50 epochs)
Phase 2 (Video-based)	6 hours (18,169 samples, 44 epochs)
Peak memory usage	8 GB (batch size 64)

A.2 Code Repository Structure

```
1 iti123_v2/
2 --- data/
3 |   --- clips/                                # 22,302 video clips (5 shot types)
4 | |   --- Clear/                             # 2,664 clips
5 | |   --- Drive/                             # 630 clips
6 | |   --- Drop/                             # 5,773 clips
7 | |   --- Lift/                             # 5,230 clips
8 | |   --- Smash/                            # 3,872 clips
9 |   --- poses/                              # MediaPipe pose extractions (Phase 1)
10 |   --- metadata.csv                       # Shot annotations with timing
11 |
12 --- notebooks/
13 |   --- badminton_action_recognition_training.ipynb # Phase 1
14 |   --- badminton_video_classification.py          # Phase 2
15 |
16 --- scripts/
17 |   --- extract_shuttleset_clips.py              # Video clip extraction
18 |   --- extract_poses_parallel.py                # Parallel pose extraction
19 |   --- validate_video_clips.py                  # Data quality checks
20 |
21 --- shot_coach/                                # Phase 3 application
```

```

22 |   +-- app.py                                # Streamlit web interface
23 |   +-- modules/
24 |   |   +-- shot_classifier.py                # ResNet18+BiLSTM inference
25 |   |   +-- pose_extractor.py                 # MediaPipe wrapper
26 |   |   +-- technique_analyzer.py            # Biomechanics (disabled)
27 |   +-- test_shot_coach.py                    # CLI testing script
28 |   +-- requirements.txt                       # Python dependencies
29 |   +-- README.md                             # User documentation
30 |   +-- README_CLASSIFIER.md                  # Technical documentation
31 |   +-- VIDEO_METADATA_FEATURE.md             # Feature specification
32 |
33 +-- outputs/
34 |   +-- results_optionA/                     # Phase 2 training results
35 |   |   +-- best_model.pth                   # Trained model (164 MB, Git LFS)
36 |   |   +-- results_summary.json             # Training metrics
37 |   |   +-- classification_report.txt        # Per-class performance
38 |   |   +-- confusion_matrix.csv             # Detailed confusion matrix
39 |   +-- reports/
40 |       +-- ITI123_Milestone_Report.tex      # Phase 1 report
41 |       +-- ITI123_Final_Project_Report.tex  # This document
42 |
43 +-- docs/                                    # Documentation
44 |   +-- VIDEO_TRAINING_README.md              # Training guide
45 |   +-- COLAB_SETUP_GUIDE.md                 # Cloud training setup
46 |   +-- SHUTTLESET_EXTRACTION_GUIDE.md        # Data preprocessing

```

Listing 8: Project Directory Organization

A.3 Deployment Commands

A.3.1 Phase 1: Pose-Only Training

```

1 # 1. Extract poses from videos (parallel processing)
2 python scripts/extract_poses_parallel.py \
3     --videos data/clips \
4     --output data/poses \
5     --workers 4 \
6     --batch-size 100
7
8 # Output:
9 # Processed 4,655 videos in 12 hours
10 # Success rate: 99.4%
11 # Saved to data/poses/*.pkl
12
13 # 2. Train pose-based models
14 cd notebooks
15 jupyter notebook badminton_action_recognition_training.ipynb
16
17 # Run all cells
18 # Models trained: ResNet1D, LSTM, BiLSTM, GRU, Transformer
19 # Result: All ~50% accuracy (failed)

```

Listing 9: Extract Poses and Train Models

A.3.2 Phase 2: Video-Based Training

```
1 # Train video-based model
2 cd notebooks
3 python badminton_video_classification.py
4
5 # Training progress (selected epochs):
6 # Epoch 10/50 - Train: 61.2%, Val: 58.5%, Loss: 1.124
7 # Epoch 20/50 - Train: 72.5%, Val: 68.3%, Loss: 0.812
8 # Epoch 30/50 - Train: 79.1%, Val: 73.2%, Loss: 0.651
9 # Epoch 40/50 - Train: 82.9%, Val: 75.4%, Loss: 0.542 [Best]
10 # Epoch 44/50 - Early stopping triggered
11 #
12 # Final Results:
13 # Test Accuracy: 74.62%
14 # Test F1-Score: 0.7361 (macro)
15 # Model saved: outputs/results_optionA/best_model.pth (164 MB)
```

Listing 10: Train ResNet18+BiLSTM on Videos

A.3.3 Phase 3: Shot Coach Deployment

```
1 # 1. Install dependencies
2 cd shot_coach
3 pip install -r requirements.txt
4
5 # Dependencies installed:
6 # streamlit==1.29.0
7 # torch==2.0.1
8 # torchvision==0.15.2
9 # opencv-python==4.8.1.78
10 # numpy==1.24.3
11 # Pillow==10.1.0
12
13 # 2. Run web application
14 streamlit run app.py
15
16 # Output:
17 # You can now view your Streamlit app in your browser.
18 # Local URL: http://localhost:8501
19 # Network URL: http://192.168.1.100:8501
20
21 # 3. Alternative: Command-line testing
22 python test_shot_coach.py ../data/clips/Smash/video.mp4
23
24 # Output:
25 # =====
26 # SHOT COACH TEST
27 # =====
28 #
29 # Video: video.mp4
30 # Loading models...
31 # [OK] Models loaded
32 #
33 # Step 1: Classifying shot type...
```

```

34 # [OK] Shot Type: Smash (94.3% confidence)
35 #
36 # Video Analysis Info:
37 #   Duration: 2.47s
38 #   Total frames: 74
39 #   FPS: 30.0
40 #   Frames analyzed: 16
41 #   Timeframe: 0.00s - 2.43s
42 #
43 # =====
44 # TEST COMPLETE
45 # =====

```

Listing 11: Run Shot Coach Application

A.4 Sample Application Output

A.4.1 Shot Coach Web Interface Output

```

=====
BADMINTON SHOT CLASSIFIER
AI-Powered Shot Type Recognition
=====

[OK] Model loaded successfully!

-----

How to Use (click to expand)

Instructions:
1. Record a 2-3 second video of ONE badminton shot
2. Upload the video using the button below
3. Wait for classification (takes 5-10 seconds)
4. See which shot type was detected!

Supported Shot Types (5 classes):
- * Clear - High defensive shot to the back
- Drive - Fast, flat attacking shot
- Drop - Soft shot over the net
- Lift - Defensive lob to the back
- * Smash - Powerful downward attack

Video Requirements:
- [WARNING] ONE shot per video (model samples frames across entire video)
- Clear view of shot execution (side-court angle works best)
- 2-3 seconds duration (prep -> contact -> follow-through)
- Full body visible in frame
- Good lighting

```

- MP4, AVI, or MOV format

Upload Your Shot Video

[Browse Files] Choose a video file (MP4, AVI, MOV)

[Uploaded: smash_example.mp4 (2.8 MB)]

[Analyze Shot]

Processing...

.....: 100%

[OK] Classification complete!

=====

* SHOT CLASSIFICATION RESULTS

=====

Detected Shot: Smash

Confidence: 94.3%

[#####] 94.3%

Video Analysis Info

Video Duration: 2.47s Total Frames: 74 Frames Analyzed: 16

Analyzed timeframe: 0.00s - 2.43s (frames sampled uniformly)

[OK] High confidence - video angle matches training data well!

All Shot Type Probabilities

* Smash (Predicted)	[#####]	94.3%
Clear	[####.....]	4.2%
Drive	[.....]	1.1%
Drop	[.....]	0.3%
Lift	[.....]	0.1%

Shot Type Information

[INFO] * Smash - Powerful attacking shot hit downward. The primary attacking shot in badminton.

* Alternative Possibilities

- Clear: 4.2% probability

* Clear - High defensive shot to the back of the court.

(Other probabilities < 1%)

[INFO] Model Information

- Model: ResNet18 + BiLSTM
- Accuracy: 74.6% (on test set)
- Classes: 5 badminton shot types
- Training: 22,302 video samples

[INFO] Note: Technique analysis (biomechanics, form scoring) is currently disabled. This version focuses on accurate shot classification only.
